Taller 7 — Tarea C: El perseguidor

Pixel Dungeon Lite · Desarrollo de videojuegos 2D y 3D · Corte 2 · Sesión 7 · Grupo 10

1. Datos del estudiante

- **Nombre completo:** Bryan Smith Mejia Poveda
- **Usuario de GitHub:** bryanmejia0410
- **Repositorio del equipo:** https://github.com/Tairacx/Pixel-dungeon-lite

2. Mi tarea

Tarea C — El perseguidor (receta C: otro personaje que se mueve solo). Archivos propios: `perseguidor.tscn` y `perseguidor.gd`.

3. Mis commits

Nivel	Mensaje	Enlace al commit
1	T7 N1 - Tarea C - escena del enemigo	https://github.com/Tairacx/Pixel-dungeon-lite/commit/8a26c1ba444b9c98d1ae5dff3994a3806b7ff5ee
2	T7 N2 - Tarea C - sigue al jugador	https://github.com/Tairacx/Pixel-dungeon-lite/commit/f7e1fe64a0f104b0c52e71ac0df6728970fd55e3
3	T7 N3 - Tarea C - patrulla y persecucion	https://github.com/Tairacx/Pixel-dungeon-lite/commit/d81a9ef1dfc49e68a70f1de8b87c6576f466dd7b

4. Nivel 1 — La escena del enemigo

Antes de programar

1. **Distancia inicial en el suelo:** $\sqrt{(6^2 + 6^2)} = \sqrt{72} =$ **8.49 m**.
2. **Con el Personaje en (0, 0, −3):** $\sqrt{(6^2 + 9^2)} = \sqrt{117} =$ **10.82 m**.

3. **¿Por qué `hacia.y = 0.0`?** Si el objetivo salta, la flecha apunta hacia arriba y la distancia crece aunque esté justo encima del enemigo. Al poner y = 0 solo se mide la distancia sobre el suelo.

Lo que escribió el Output

- **Ejemplo, quieto:** `distancia 8.49` cada medio segundo, siempre igual.
- **Ejemplo, Avanzar 60 cuadros (personaje del repo):** `9.81`, `11.46` y luego `11.66` fijo. El Personaje termina en (0, 0, −3.83) en el cuadro 60 y sigue deslizando hasta cerca de z = −4.0 al soltar la tecla.
- **Mi juego (Perseguidor en (−8, 0, 6)), quieto:** `distancia 10.0` fijo.
- **Mi juego, Avanzar 60 cuadros:** `11.15`, `12.62` y luego `12.81` fijo.

Predicción vs. medida. Para mi juego predije $\sqrt{(8^2 + 6^2)}$ = **10.0 m** al inicio y, con el héroe detenido cerca de z = −4.0, $\sqrt{(8^2 + 10^2)} \approx 12.8$ m: coincidió (10.0 y 12.81). El Perseguidor se queda en su sitio, apoyado en el suelo y en idle.

5. Nivel 2 — Otro personaje que se mueve solo

Antes de programar

1. **Distancia inicial:** $\sqrt{72}$ = **8.49 m**.
2. **Tiempo hasta 1 m:** (8.49 − 1) ÷ 2 = **3.74 s**, es decir unos **225 cuadros**. En 30 cuadros (0.5 s) recorre 2 × 0.5 = 1 m, así que la distancia baja **1 m cada 30 cuadros**.
3. **Si no se detuviera a 1 m:** cuando están casi encima, la flecha es diminuta y `normalized()` la deja apuntando hacia un lado distinto en cada cuadro. El enemigo vibraría y empujaría al héroe. Con el `if distancia > 1.0` eso no pasa.

Lo que escribió el Output

- **Ejemplo (2 m/s):** `7.52, 6.52, 5.52, 4.52, 3.52, 2.52, 1.52` y desde ahí `0.99` fijo. Se detiene un poco antes de 1 m (por el paso de 2 ÷ 60 = 0.033 m por cuadro) y queda en idle. Coincide con lo que dice la guía.
- **Mi juego (2.5 m/s, desde (−8, 0, 6), héroe quieto en el origen):** `8.79, 7.54, 6.29, 5.04, 3.79, 2.54, 1.29` y luego `0.96` fijo.

Predicción vs. medida. Con 2.5 m/s la distancia debía bajar 2.5 × 0.5 = **1.25 m cada 30 cuadros** y llegar a 1 m a los (10 − 1) ÷ 2.5 = **3.6 s (216 cuadros)**. Las líneas del Output bajan de a 1.25 (8.79 → 7.54 → 6.29 → …) y se detiene en 0.96, así que la predicción coincidió. La distancia final queda un poco por debajo de 1.0 porque el último paso es de 2.5 ÷ 60 = 0.042 m.

6. Nivel 3 — Patrullar, ver y perseguir

Antes de programar

¿En qué x queda a 6 m del Personaje? En la línea z = −3: $x^2 + 9 = 36$, $x^2 = 27$, x = ±**5.2**. Como el guardia sale de x = −6, **no alcanza a llegar a x = 0 sin verlo**: lo ve cuando ha avanzado unos 0.8 m (unos 0.4 s).

1. **Huye a 5 m/s y lo siguen a 2 m/s:** la distancia crece a 5 − 2 = 3 m/s, así que pasa de 6 m a 9 m en 3 ÷ 3 = **1 s**.
2. **Con un solo radio de 6 m y el Personaje justo a 6 m:** el guardia daría un paso hacia adelante (5.99 m: lo ve) y otro hacia atrás (6.01 m: lo pierde), y cambiaría de estado en cada cuadro. Por eso el radio de perder es más grande (histéresis).
3. **Para mi juego:** ve a **8 m** (256 px), lo pierde a **12 m** (384 px) y patrulla entre (−8, 0, 6) y (8, 0, 6) a 2.5 m/s.

Lo que escribió el Output

- **Ejemplo (6 m / 9 m, héroe quieto en el origen):** `lo vio a 5.97 m` en el primer medio segundo. Moviendo al héroe de golpe a (0, 0, 12) después de unos cuadros: `lo perdió a 13.84 m`.
- **Mi juego (8 m / 12 m, héroe quieto en el origen):** `lo vio a 7.97 m`. Con el héroe movido de golpe a (0, 0, 14) en el cuadro 200: `lo perdió a 12.26 m` y vuelve a patrullar.

Predicción vs. medida. En mi juego, el guardia ve al héroe cuando $x^2 + 36 < 64$, o sea en x = −5.29. Parte de x = −8, así que recorre 2.71 m a 2.5 m/s: unos 1.08 s, cerca del cuadro 65. El Output da 7.97 m, apenas por debajo de 8 porque se mide en pasos de 0.042 m por cuadro. La predicción coincidió. Para "lo perdió", el Output dio un número apenas mayor que 12, como debe ser con un radio de 12 m.

7. Tabla de números del juego

Qué es	Número	¿Ya estaba o lo propuse?	Por qué ese número	Lo que midió la prueba
Vida del perseguidor	3 HP	Ya estaba en el documento	Se copia tal cual	
Daño al héroe	1 corazón	Ya estaba en el documento	Se copia tal cual	
Rapidez del perseguidor	80 px/s = 2.5 m/s	Ya estaba en el documento	Con 32 px = 1 m, 80 px/s son 2.5 m/s; el héroe camina a 4 m/s y corre a 6 m/s, así que siempre puede alejarse	La distancia baja 1.25 m cada 30 cuadros; desde 10 m llega a 1 m en 3.6 s
Escala	32 px = 1 m	Propuesto	Convierte los px del documento a metros de Godot	—
Radio para ver al héroe	8 m (256 px)	Propuesto	Es un tercio del ancho de la sala (24 × 18 m en zona_a): ve al héroe en buena parte de la sala, pero no desde cualquier rincón	En mi prueba: `lo vio a 7.97 m`
Radio para perder al héroe	12 m (384 px)	Propuesto	Mayor que el de ver, para que no cambie de estado en cada cuadro en el borde. Caminando (4 m/s) el héroe lo pierde en 4 ÷ 1.5 = 2.7 s; corriendo (6 m/s), en 4 ÷ 3.5 = 1.1 s	En mi prueba: `lo perdió a 12.26 m`

Qué es	Número	¿Ya estaba o lo propuse?	Por qué ese número	Lo que midió la prueba
Enemigos por sala	3	Propuesto	Cada uno cae en 3 golpes y el héroe es invulnerable 1.5 s tras un daño, así que tres que lleguen juntos quitan como máximo 1 corazón cada 1.5 s. Cruzar la sala a 2.5 m/s toma 9.6 s, por lo que no llegan todos a la vez	Cuenta, no prueba: todavía no hay varios enemigos en la sala

8. Respuestas cortas

a. ¿Por qué la entrada del teclado se gira con la cámara y no se usa tal cual en los ejes del mundo? Porque el jugador espera que W lleve al personaje "hacia el fondo de la pantalla", sin importar hacia dónde mire la cámara. Si la cámara gira 90° y la entrada se usa en los ejes del mundo, W seguiría llevando a −Z y en pantalla iría hacia un lado. Al girar la entrada con la rotación horizontal de la cámara, W siempre es "adelante" para quien mira la pantalla.

b. ¿Por qué la diagonal salía torcida con `camara.global_basis` y derecha con `rotated(Vector3.UP, ...)`? Con `global_basis` la entrada también se inclina con la cámara (que mira 35° hacia abajo): la parte de W se reparte entre "abajo" y "adelante", y al aplanar con `y = 0` la componente hacia adelante queda más corta que la lateral. Resultado: 2.32 hacia un lado y solo 1.9 hacia adelante. Con `rotated` solo se usa el giro horizontal de la cámara (la inclinación no interviene), así que W y D pesan igual y los dos lados miden 2.12 m: una diagonal de 45°.

c. ¿Qué vacío llenó mi tarea y qué número propuse? El documento no decía cuántos enemigos hay por sala ni a qué distancia ven al héroe. Propuse **3 enemigos por sala**, ver a **8 m** y perder a **12 m**. Elegí 8 m porque es un tercio del ancho de la sala de 24 m; 12 m porque debe ser mayor que 8 m (histéresis) y, con el héroe a 4 m/s contra 2.5 m/s del perseguidor, le da a un jugador que camina unos 2.7 s para escapar y a uno que corre solo 1.1 s. Con 3 enemigos hay presión, pero la invulnerabilidad de 1.5 s evita que se pierdan los corazones de golpe.

d. Cuando lo probé en el juego, ¿el número se sintió bien? [Escribe aquí cómo se sintió al jugarlo con F5: ¿te parece justo que te vea a 8 m?]. Con la medida de la prueba: el perseguidor va 1.5 m/s más lento que el héroe caminando y 3.5 m/s más lento corriendo; los radios dan un escape en 2.7 s caminando. Si al jugarlo se siente demasiado fácil, subiría la rapidez del perseguidor a 3 m/s (96 px/s): la diferencia caminando bajaría a 1 m/s y el escape tomaría 4 s. Si se siente muy difícil, bajaría el radio de ver a 6 m (192 px).